

Быстрая сортировка

Материал из Википедии — свободной энциклопедии

1 Текущая версия страницы пока не проверялась опытными участниками и может значительно отличаться от версии, проверенной 17 января 2020 года; проверки требуют 86 правок.	
---	--

Быстрая сортировка, **сортировка Хоара** (англ. quicksort), часто называемая ***qsort*** (по имени в стандартной библиотеке языка Си) — алгоритм сортировки, разработанный английским информатиком Тони Хоаром во время его работы в МГУ в 1960 году.

Один из самых быстрых известных универсальных алгоритмов сортировки массивов: в среднем ***O**(n log n)* обновов при упорядочении *n* элементов; из-за наличия ряда недостатков на практике обычно используется с некоторыми доработками.

Содержание [скрыть]
1 Общее описание 2 Алгоритм 2.1 Общий механизм сортировки 2.2 Выбор опорного элемента 2.3 Разбиение Ломута 3 Схема Хоара 3.1 Повторяющиеся элементы 4 Оценка сложности алгоритма 5 Достоинства и недостатки 6 Улучшения 7 См. также 8 Примечания 9 Литература 10 Ссылки

Общее описание

1 В разделе не хватает ссылок на источники (см. рекомендации по поиску). Информация должна быть проверяема, иначе она может быть удалена. Вы можете отредактировать статью, добавив ссылки на авторитетные источники в виде сносок. (<i>21 февраля 2023</i>)	
--	--

QuickSort является существенно улучшенным вариантом алгоритма сортировки с помощью прямого обмена (его варианты известны как «Пузырьковая сортировка» и «Шейкерная сортировка»), известного в том числе своей низкой эффективностью. Принципиальное отличие состоит в том, что в первую очередь производится перестановка на наибольшее возможном расстоянии и после каждого прохода элементы делятся на две независимые группы (таким образом улучшение самого неэффективного прямого метода сортировки дало в результате один из наиболее эффективных улучшенных методов).

Общая идея алгоритма состоит в следующем:

- Выбрать из массива элемент, называемый опорным. Это может быть любой из элементов массива. От выбора опорного элемента не зависит корректность алгоритма, но в отдельных случаях может сильно зависеть его эффективность (см. ниже).
- Сравнить все остальные элементы с опорным и переставить их в массиве так, чтобы разбить массив на три непрерывных отрезка, следующих друг за другом: «элементы меньше опорного», «равные» и «большие»^[a].
- Для отрезков «меньших» и «больших» значений выполнить рекурсивно ту же последовательность операций, если длина отрезка больше единицы.

На практике массив обычно делят не на три, а на две части: например, «меньше опорного» и «равные и больше»; такой подход в общем случае эффективнее, так как упрощает алгоритм разделения (см. ниже).

Хоар разработал этот метод применительно к **машинному переводу**: словарь хранился на **магнитной ленте**, и сортировка слов обрабатываемого текста позволяла получить их поведы за один прогон ленты, без перемоти её назад. Алгоритм был придуман Хоаром во время его пребывания в **Советском Союзе**, где он обучался в **Московском университете** компьютерному переводу и занимался разработкой русско-английского разговорника.

Алгоритм [править] [править код]

Общий механизм сортировки [править] [править код]

Быстрая сортировка относится к алгоритмам «разделяй и властвуй».

Алгоритм состоит из трёх шагов:

- Выбрать элемент из массива. Назовём его опорным.
- Разбиение**: перераспределение элементов в массиве таким образом, что элементы, меньшие опорного, помещаются перед ним, а большие или равные — после.
- Рекурсивно перейти первые два шага к двум подмассивам слева и справа от опорного элемента. Рекурсия не применяется к массиву, в котором только один элемент или отсутствуют элементы.

В наиболее общем виде алгоритм на **псевдокоде** (где *A* — сортируемый массив, а low и high — соответственно, нижняя и верхняя границы сортируемого участка этого массива) выглядит следующим образом:

algorithm quicksort(<i>A</i>, low, high) is if low < high then p := partition(<i>A</i>, low, high) quicksort(<i>A</i>, low, p - 1) quicksort(<i>A</i>, p, high)	
--	--

Здесь предполагается, что массив *A* передаётся по ссылке, то есть сортировка происходит «на том же месте», а неописанная функция partition возвращает индекс опорного элемента.

Для выбора опорного элемента и операции разбиения существуют разные подходы, влияющие на производительность алгоритма.

Возможна также следующая реализация быстрой сортировки:

algorithm quicksort(<i>A</i>) is if <i>A</i> is empty return <i>A</i> pivot:= <i>A</i>.pop() (извлечь последний или первый элемент из массива) lA:= <i>A</i>.filter(where e <= pivot) (создать массив с элементами меньше опорного) rA := <i>A</i>.filter(where e > pivot) (создать массив с элементами больше опорного) return quicksort(lA) + [pivot] + quicksort(rA) (вернуть массив, состоящий из отсортированной левой части, опорного и отсортированной правой части).	
--	--

На практике она не используется, а служит лишь в образовательных целях, так как использует дополнительную память, чего можно избежать.

Выбор опорного элемента [править] [править код]

В ранних реализациях, как правило, опорным выбирался первый элемент, что снижало производительность на отсортированных массивах. Для улучшения эффективности может выбираться средний, случайный элемент или (для больших массивов) *медиана* первого, среднего и последнего элементов.^[3] Медиана всей последовательности является оптимальным опорным элементом, но её вычисление слишком трудоёмко для использования в сортировке.

Выбор опорного элемента по медиане трёх для разбиения Ломута:

mid := (low + high) / 2 if A[mid] < A[low] swap A[low] with A[mid] if A[high] < A[low] swap A[low] with A[high] if A[high] < A[mid] swap A[high] with A[mid] pivot:= A[mid]	
---	--

Разбиение Ломута [править] [править код]

Данный алгоритм разбиения был предложен Нико Ломуто^[4] и популяризован в книгах Бентли (Programming Pearls) и Кормена (Введение в алгоритмы).^[5] В данном примере опорным выбирается последний элемент. Алгоритм хранит индекс в переменной *i*. Каждый раз, когда находится элемент, меньше или равный опорному, индекс увеличивается и элемент вставляется перед опорным. После разбиения опорный элемент окажется в позиции *i* — на границе между двумя подмножествами. Хотя эта схема разбиения проще и компактнее, чем схема Хоара, она менее эффективна и используется в обучающих материалах. Сложность данной быстрой сортировки возрастает до *O*(*n*²), когда массив уже отсортирован или все его элементы равны. Существуют различные методы оптимизации данной сортировки: алгоритмы выбора опорного элемента, использование сортировки вставками на маленьких массивах. В данном примере сортируются элементы массива *A* от low до high (включительно)^[6].

algorithm partition(<i>A</i>, low, high) is pivot := A[high] i := low - 1 for j := low to high do if A[j] ≤ pivot then i := i + 1 if i ≠ j then swap A[i] with A[j] return i + 1	
---	--

Сортировка всего массива может быть выполнена с помощью выполнения quicksort(*A*, 1, length(*A*)).

Схема Хоара [править] [править код]

Данная схема использует два индекса (один в начале массива, другой — в конце), которые приближаются друг к другу, пока не найдётся пара элементов, где один больше опорного и расположен перед ним, а второй — меньше и расположен после. Эти элементы меняются местами. Обмен происходит до тех пор, пока индексы не пересекнутся. Алгоритм возвращает последний индекс.^[6] Схема Хоара эффективнее схемы Ломута, так как происходит в среднем в три раза меньше обменов (swap) элементов и разбиение эффективнее, даже когда все элементы равны.^[7] Подобно разбиению Ломута, данная схема также показывает эффективность в *O*(*n*²), когда входной массив уже отсортирован. Сортировка с использованием данной схемы нестабильна. Следует заметить, что конечная позиция опорного элемента необязательно совпадает с возвращённым индексом. **Псевдокод**^[5].

algorithm quicksort(<i>A</i>, lo, hi) is if lo < hi then p:= partition(<i>A</i>, lo, hi) quicksort(<i>A</i>, lo, p) quicksort(<i>A</i>, p + 1, hi) algorithm partition(<i>A</i>, low, high) is pivot:= A[(low + high) / 2] i:= low j:= high loop forever while A[i] < pivot i:= i + 1 while A[j] > pivot j:= j - 1 if i >= j then return j swap A[i++] with A[j--]
--

В книге^[5] упоминается, что такая реализация алгоритма имеет «много тонких моментов». Вот один из них: выбор в качестве опорного элемента уже существующего элемента в массиве может привести к переполнению стека вызовов из-за того, что номер элемента вычисляется как среднее, а это не целое число. Поэтому логичнее в данном алгоритме выбирать в качестве опорного элемента среднее значение между начальным и конечным элементом:

pivot := A[(low + high) / 2]	
---	--

Но нет никаких оснований выбирать в качестве опорного элемента A[(low + high) / 2]. Оптимальным может оказаться любой элемент от low до high с равной вероятностью. Там, где A[(low + high) / 2] — это не предыдущий раз. Такой элемент из середины массива. Гораздо разумнее выбирать случайный элемент A[random, low, high]. Это гарантирует, что никто специально не сможет соорудить массив, который этот алгоритм введёт в очень длинный цикл с переполнением стека. В этом случае вероятность неудачи при сортировке массива из примерно 37 000 элементов, по самой завышенной оценке, составляет примерно 10 в минус 16 степеней, что на 3 порядка меньше, чем вероятность выхода из строя компьютера в результате прямого попадания в него метеорита. Неудачей считается число iterations (полного перебора всех элементов массива с целью сравнения и перестановки) более чем 200.

Повторяющиеся элементы [править] [править код]

Для улучшения производительности при большом количестве одинаковых элементов в массиве может быть применена процедура разбиения массива на три группы: элементы меньше опорного, равные ему и больше него (Бентли и Макилрой называют это «толстым разбиением». Данное разбиение используется в функции *qsort* в седьмой версии Unix^[8]. Псевдокод:

algorithm quicksort(<i>A</i>, low, high) is if low < high then p := pivot(<i>A</i>, low, high) left, right := partition(<i>A</i>, p, low, high) <i>// возвращаетс два значения</i> quicksort(<i>A</i>, low, left) quicksort(<i>A</i>, right, high)	
--	--

Быстрая сортировка используется в стандарте языка C++, в частности, в методе sort структуры данных list

1 # include <iostream> 2 # include <list> 3 4 int main() 5 { 6 <i> // quick sort</i> 7 <i>std::list<int></i> list; 8 <i>const</i> <i>int</i> N = 20; 9 <i>for</i> (<i>int</i> i = 0; i < N; i++) 10 { 11 <i>if</i> (i % 2 == 0) 12 <i>list.push_back</i>(N - i); 13 <i>else</i> 14 <i>list.push_front</i>(N - i); 15 } <i>for</i> (<i>std::list<int></i>::iterator it = list.begin(); it != list.end(); it++) { 16 <i>std::cout</i> << (*it) << " "; 17 } <i>std::cout</i> << <i>std::endl</i>; 18 <i>list.sort</i>(); 19 <i>for</i> (<i>std::list<int></i>::iterator it = list.begin(); it != list.end(); it++) { 20 <i>std::cout</i> << (*it) << " "; 21 } 22 <i>std::cout</i> << <i>std::endl</i>; 23 <i>//quick sort end</i> 24 <i>std::cout</i> << <i>std::endl</i>; 25 <i>//sort greater</i> 26 <i>list.sort</i>(<i>std::greater<int></i>()); 27 <i>for</i> (<i>std::list<int></i>::iterator it = list.begin(); it != list.end(); it++) { 28 <i>std::cout</i> << (*it) << " "; 29 } 30 <i>std::cout</i> << <i>std::endl</i>; 31 <i>//sort greater end</i> 32 <i>std::cin.ignore</i>(); 33 34 } 35
--

Оценка сложности алгоритма [править] [править код]

Люю, что операция разделения массива на две части относительно опорного элемента занимает время ***O**(n)*. Поскольку все операции разделения, продлеваемые на одной глубине рекурсии, обрабатывают разные части исходного массива, размер которого постояен, суммарно на каждом уровне рекурсии потребуется также ***O**(n)* операций. Следовательно, общая сложность алгоритма определяется лишь количеством разделений, то есть глубиной рекурсии. Глубина рекурсии, в свою очередь, зависит от сочетания входных данных и способа определения опорного элемента.

Лучший случай.

В наиболее сбалансированном варианте при каждой операции разделения массив делится на две одинаковые части, следовательно, максимальная глубина рекурсии, при которой размеры обрабатываемых подмассивов достигнут 1, составит **log**₂ *n*. В результате количество сравнений, совершаемых быстрой сортировкой, было бы равно значению рекурсивного выражения *C*_н = 2 · *C*_{n/2} + *n*, что даёт общую сложность алгоритма ***O**(n · log*₂ *n)*.

Среднее.

Среднее сложность при случайном распределении входных данных можно оценить той вероятностью. Прежде всего необходимо заметить, что в действительности необязательно, чтобы опорный элемент всяко раз делил массив на две *одинаковых* части. Например, если на каждом этапе будет происходить разделение на массивы длиной 75 % и 25 % от исходного, глубина рекурсии будет равна **log**_{4/3} *n*, а это по-прежнему даёт сложность ***O**(n log n)*. Вообще, при любом *фиксированном* соотношении между левой и правой частями разделения сложность алгоритма будет той же, только с разными константами.

Будем считать «удачным» разделениеи такое, при котором опорный элемент окажется среди центральных 50 % элементов разделяемой части массива; ясно, что вероятность удачи при случайном распределении элементов составляет 0,5. При удачном разлении размеры выделенных подмассивов составят не менее 25 % и не более 75 % от исходного. Поскольку каждый выделенный подмассив так же будет иметь случайное распределение, все эти рассуждения применимы к любому этапу сортировки и любому исходному фрагменту массива.

Удачное разделение даёт глубину рекурсии не более **log**_{4/3} *n*. Поскольку вероятность удачи равна 0,5 для получения *k* удачных разделений, средню потребуется **2 · k** рекурсивных вызовов, чтобы опорный элемент *k* раз оказался среди центральных 50 % подмассива. Применяя эти соображения, можно заключить, что в среднем глубина рекурсии не превысит **2 · log**_{4/3} *n*, что равно ***O**(log n)* *A*, поскольку на каждом уровне рекурсии по-прежнему выполняется не более ***O**(n)* операций, средняя сложность составит ***O**(n log n)*.

Худший случай.

В самом несобалансированном варианте каждое разделение даёт два подмассива размерами 1 и *n* — 1, то есть при каждом рекурсивном вызове больший массив будет на 1 короче, чем в предыдущий раз. Такое может произойти, если в качестве опорного на каждом этапе будет выбран элемент либо наименьший, либо наибольший из всех обрабатываемых. При простейшем выборе опорного элемента — первого или последнего в массиве — такой эффект даст уже отсортированный (в прямом или обратном порядке) массив, для среднего или любого другого фиксированного элемента «массив худшего случая» также может быть специально подобран. В этом случае потребуется *n* — 1 операция разделения, а общее время работы составит ∑*n* − 1 (*n* — 1) = ***O**(n²)* операций, то есть сортировка будет выполняться за квадратичное время. Но количество обменов и, соответственно, время работы — это не самый большой его недостаток. Хуже то, что в таком случае глубина рекурсии при выполнении алгоритма достигнет *n*, что будет означать *n*-кратное сохранение адреса возврата и локальных переменных процедуры разделения массивов. Для больших значений *n* худший случай может привести к исчерпанию памяти (**переполнению стека**) во время работы программы.

Достоинства и недостатки [править] [править код]

Достоинства:

- Один из самых быстройдействующих (на практике) из алгоритмов внутренней сортировки, неבעлого назначения.
- Алгоритм очень короткий: запустив основные моменты, его легко написать «из головы», небега константа при ***n** log n*.
- С модификациями требует лишь ***O**(log n)* дополнительной памяти в виде стека (неулучшенной рекурсивный алгоритм — в худшем случае ***O**(n)* стека).
- Хорошо сочетается с механизмами *каширования* и *виртуальной памяти*.
- Допускает естественное распараллеливание (сортировка выделенных подмассивов в параллельно выполняющихся подпроцессах).
- Допускает эффективную модификацию для сортировки по нескольким ключам (в частности, алгоритм *Седжика* для сортировки строк); благодаря тому, что в процессе разделения автоматически выделяется отрезок элементов, в частности **это отрезок** можно сразу же сортировать по следующему ключу.
- Работает на *связных списках* и других структурах с последовательным доступом, допускающих эффективный проход как от начала к концу, так и от конца к началу.

Недостатки:

- Сильно деградирует по скорости (до ***O**(n²)*) в худшем или близком к нему случае, что может случиться при неудачных входных данных.
- Прямая реализация в виде функции с двумя рекурсивными вызовами может привести к ошибке *переполнения стека*, так как в худшем случае ей может потребоваться сделать ***O**(n)* вложенных рекурсивных вызовов.
- Неустойчив*.

Улучшения [править] [править код]

Улучшения алгоритма направлены, в основном, на устранение или смягчение вышеупомянутых недостатков, вследствие чего все их можно разделить на три группы: придание алгоритму устойчивости, устранение деградации производительности специальным выбором опорного элемента, и защита от переполнения стека вызовов из-за большой глубины рекурсии при неудачных входных данных.

- Проблема неустойчивости решается путём расширения ключа исходным индексом элемента в массиве. В случае равенства основных ключей сравнение производится по индексу, исключая, таким образом, возможность изменения взаимного положения равных элементов. Эта модификация не бесплатна — она требует дополнительно *O*(n) памяти и одного полного прохода по массиву для сохранения исходных индексов.
- Деградация по скорости в случае неудачного набора входных данных решается по двум разным направлениям: снижение вероятности возникновения худшего случая путём специального выбора опорного элемента и применение различных технических приёмов, обеспечивающих устойчивую работу на неудачных входных данных. Для первого направления:

- Выбор среднего элемента. Устраняет деградацию для предварительно отсортированных данных, но оставляет возможность случайного появления или намеренного подбора «плохого» массива.
- Выбор медианы из трёх элементов: первого, среднего и последнего. Снижает вероятность возникновения худшего случая, по сравнению с выбором среднего элемента.
- Случайный выбор. Вероятность случайного возникновения худшего случая становится исчезающе малой, а намеренный подбор — практически неосуществимым. Ожидаемое время выполнения алгоритма сортировки составляет *O*(n log *n*).

Недостаток всех усложнённых методов выбора опорного элемента — дополнительные накладные расходы; впрочем, они не так велики.

- Во избежание отказа программы из-за большой глубины рекурсии могут применяться следующие методы:

- При достижении нежелательной глубины рекурсии переходить на сортировку другими методами, не требующими рекурсии. Примером такого подхода является алгоритм *IntroSort* или некоторые реализации быстрой сортировки в библиотеке *STL*. Можно заметить, что алгоритм очень хорошо подходит для такого рода модификаций, так как на каждом этапе предлагается выделить непрерывный отрезок исходного массива, предназначенный для сортировки, и то, каким методом будет отсортирован этот отрезок, никак не влияет на обработку остальных частей массива.
- Модификация алгоритма, устраняющая одну ветвь рекурсии: вместо того, чтобы после разделения массива вызывать рекурсивно процедуру разделения для обоих найденных подмассивов, рекурсивный вызов делается только для меньшего подмассива, а больший обрабатывается в **цикле в пределах этого же вызова процедуры**. С точки зрения эффективности в среднем случае разницы практически нет: накладные расходы на дополнительный рекурсивный вызов и на организацию сравнения длин подмассивов и цикла — это примерно столько же. Зато глубина рекурсии ни при каких обстоятельствах не превысит **log**₂ *n*, а в худшем случае вырожденного разделения она вообще будет не более 2 — вся обработка пройдёт в цикле первого уровня рекурсии. Применение этого метода не спасёт от катастрофического падения производительности, но переполнения стека не будет.

- Разбивать массив не на две, а на три части^[9].

См. также [править] [править код]

- Список алгоритмов сортировки

Примечания [править] [править код]

- ↑ Hoare C. A. R. *Algorithm 64: Quicksort* (англ.) // *Communications of the ACM* — New York City: Association for Computing Machinery, 1961. — Vol. 4, Iss. 7. — P. 321. — ISSN 0001-0782 [: 1557-7317] — doi:10.1145/366622.366644
- ↑ Очевидно, что после такой перестановки для получения отсортированного массива не понадобится перемещать ни один из элементов между получившимися отрезками, то есть достаточно будет произвести сортировку «меньшего» и «большего» отрезков как самостоятельных массивов.
- ↑ *Sedgewick, Robert* *Algorithms in C: Fundamentals, Data Structures, Sorting, Searching, Parts 1-4* (англ.). — 3. — Pearson Education^[англ.]. 1998. — ISBN 978-81-317-1291-7.
- ↑ Jon Bentley. Programming Pearls (англ.). — Addison-Wesley Professional, 1999.
- 1 2 3 *Кормен, Т., Лейзерсон, Ч., Рувест, Р., Штайн, К.* *Quicksort* /</